Sad pet

Finally, make Screen Pet notice if you don't pay any attention to it. After nearly a minute without being stroked, your poor, neglected pet will show its sad face!

21 Set up a happiness level

Put this line of code just above the flag variables you added to the main part of the program in Step 16. It creates a happiness level for Screen Pet and sets the level at 10 when you run the program and draw the pet.

```
c.happy_level = 10
c.eyes_crossed = False
```

Screen Pet starts with a happiness level of 10.

22 Create a new command

Type this line below the command you added in Step 9 that starts Screen Pet blinking. It tells **`mainloop()`** to call the function **`sad()`**, which you'll add in Step 23, after 5 seconds (5,000 milliseconds).

```
root.after(1000, blink)
root.after(5000, sad)
root.mainloop()
```

23 Write a sad function

Add this function, **`sad()`**, beneath **`hide_happy()`**. It checks to see if **`c.happy_level`** is 0 yet. If it is, it changes Screen Pet's expression to a sad one. If it's not, it subtracts 1 from **`c.happy_level`**. Like **`blink()`**, it reminds **`mainloop()`** to call it again after 5 seconds.

```
def sad():
    if c.happy_level == 0:
        c.itemconfigure(mouth_happy, state=HIDDEN)
        c.itemconfigure(mouth_normal, state=HIDDEN)
        c.itemconfigure(mouth_sad, state=NORMAL)
    else:
        c.happy_level -= 1
    root.after(5000, sad)
```

This line checks to see if the value of **`c.happy_level`** is 0.

If **`c.happy_level`** equals 0, the code hides the happy and normal expressions.

This line sets Screen Pet's expression to sad.

The value of **`c.happy_level`** is greater than 0 (**`else`**).

Subtract 1 from the value of **`c.happy_level`**.

Call **`sad()`** again after 5,000 milliseconds.